

Final DevSecOps Report

1. Project Presentation

This project presents a complete DevSecOps delivery chain for a Zero-Trust Vault application. The application is composed of a FastAPI backend, a React frontend, an ELK monitoring stack, and a SecOps automation bot connected to GitLab, Elasticsearch, Telegram, and an LLM provider.

The objective is to demonstrate how security can be integrated into each phase of software delivery: code validation, static analysis, container build, image scanning, dynamic application testing, deployment, monitoring, and automated incident assistance.

2. DevSecOps Architecture

Layer	Component	Role
Application	FastAPI backend	Provides authentication and protected secret retrieval endpoints.
Application	React frontend	Provides the user interface for login and secure data access.
CI/CD	GitLab CI/CD	Orchestrates quality, security, build, scan, and deployment jobs.
SAST	SonarQube / SonarScanner	Detects code smells, bugs, vulnerabilities, and maintainability issues.
Container security	Trivy	Scans Docker images for high and critical vulnerabilities.

DAST	OWASP ZAP	Tests the running web application from an external attacker perspective.
Monitoring	Filebeat, Elasticsearch, Kibana	Collects, indexes, and visualizes runtime logs.
Automation	SecOps bot	Queries SIEM data and GitLab pipelines, then summarizes incidents.
Deployment	Docker Compose / Kubernetes	Runs the stack locally or in a cluster environment.

3. CI/CD Pipeline Explanation

The global GitLab pipeline contains nine stages: install, lint, test, SAST, build, container scan, push, DAST, and deploy.

3.1 Install

The install stage prepares Python dependencies for the backend and validates that dependency files are usable inside CI.

3.2 Lint

The lint stage checks backend code with `flake8` and frontend code with `npm run lint`.

3.3 Test

The test stage runs backend tests with `pytest` and exports Cobertura coverage.

3.4 SAST

The SAST stage runs SonarScanner against backend, frontend, and SecOps bot source code.

3.5 Build

The build stage creates Docker images for the backend, frontend, and SecOps bot.

3.6 Container Scan

The container scan stage uses Trivy to inspect the generated Docker image artifacts for high and critical vulnerabilities.

3.7 Push

The push stage uploads images to GitLab Container Registry when the pipeline runs on the main branch.

3.8 DAST

The DAST stage runs OWASP ZAP baseline scanning against the deployed frontend.

3.9 Deploy

The deploy stage documents Docker Compose and Kubernetes deployment options.

4. Tools Used

Tool	Category	Usage
GitLab CI/CD	CI/CD orchestration	Executes all pipeline stages.
Docker	Containerization	Packages backend, frontend, and bot services.
Docker Compose	Local deployment	Runs the full stack on one host.
Kubernetes	Cluster	Defines deployments and

	deployment	services for the stack.
Flake8	Linting	Checks Python formatting and style issues.
Pytest	Testing	Runs backend tests and exports coverage.
SonarScanner	SAST	Performs static application security testing.
Trivy	Container scanning	Finds vulnerabilities in container images.
OWASP ZAP	DAST	Performs dynamic web application security testing.
Filebeat / Elasticsearch / Kibana	Monitoring	Collects, indexes, and visualizes runtime logs.
Telegram bot and LLM API	Automation	Summarizes SIEM and pipeline events.

5. Security Scan Result Analysis

5.1 SAST Findings

The backend contains a hardcoded JWT secret, permissive CORS, and demo credentials. These are acceptable for a classroom demonstration but must be remediated before production deployment.

The frontend contains a hardcoded API URL. This should be injected through environment-specific configuration.

5.2 Container Scan Findings

Trivy is configured to report high and critical vulnerabilities. The remediation process is to update base images, update dependencies, rebuild, rescan, and document

accepted risks.

5.3 DAST Findings

OWASP ZAP checks the running frontend for missing headers, passive scan alerts, exposed endpoints, and weak browser security configuration.

5.4 Monitoring Findings

Filebeat sends container logs to Elasticsearch. The SecOps bot watches for repeated 401 and error events and can summarize incidents through Telegram.

6. Screenshots

Screenshot	Evidence
01-gitlab-pipeline-overview.png	Complete GitLab CI/CD pipeline graph.
02-sonarqube-dashboard.png	SAST dashboard and quality gate.
03-trivy-scan-results.png	Container vulnerability scan output.
04-zap-dast-report.png	OWASP ZAP HTML report.
05-kibana-dashboard.png	Runtime logs indexed in Kibana.
06-docker-compose-services.png	Local stack running with Docker Compose.
07-kubernetes-workloads.png	Kubernetes pods/services in running state.
08-secops-bot-alert.png	Telegram bot alert or command response.

7. Deployment

7.1 Docker Compose

```
docker compose up -d
```

7.2 Kubernetes

```
kubect1 apply -k kubernetes/
```

Sensitive values for the SecOps bot are represented as Kubernetes secrets and must be replaced before deployment.

8. Conclusion

This project demonstrates a complete DevSecOps workflow across the full software delivery lifecycle. The CI/CD pipeline validates the code, performs static analysis, builds Docker images, scans containers, runs dynamic security testing, and prepares deployment. The monitoring stack and SecOps bot extend the project beyond deployment by adding runtime visibility and automated incident assistance.

The main improvement areas are secret management, stricter CORS configuration, stronger authentication, environment-based frontend configuration, and enforcing pipeline gates once the project moves from demonstration mode to production mode.